Web Development using PHP

PHP Functions

(Lecture 4)

Prepared By: BITS IT Team

Venue: GS Training Room

Table of Contents

1. PHP Functions 3
1.1 Create a PHP Function 3
Example 1.1 3
1.2 PHP Functions - Adding parameters 3
Example 1.2 4
Example 1.3 4
1.3 PHP Functions - Return values 4
Example 1.4 5
2. Using Databases 5
2.1 Step 1 (Create a Connection to the database) 5
Example 2.1 6
2.2 Step 2 (Write a query) 6
Example 2.2 6
2.3 Step 3 (Parse a query) 7
Example 2.3 7
2.4 Step 4 (Execute a query) 8
Example 2.4 8
2.5 Step 5 (Fetch the result set) 9
Example 2.5 9
2.6 Step 6 (Loop over the result set) 10
Example 2.6 10
7 Resources 12
7.1 websites 12
7.2 Books 12

1. PHP Functions

In this lecture we will show you how to create your own functions.

To keep the script from being executed when the page loads, you can put it into a function.

A function will be executed by a call to the function.

You may call a function from anywhere within a page.

1.1 Create a PHP Function

A function will be executed by a call to the function.

Syntax

function *functionName*()
{
code to be executed;
}

PHP function guidelines:

- Give the function a name that reflects what the function does
- The function name can start with a letter or underscore (not a number)

Example 1.1

A simple function that writes my name when it is called:

```
<html>
<body>
<?php
function display_name()
{
echo "My name is Zafar";
}
display_name ();
?>
</body>
</html>
```

Output:

```
My name is Zafar
```

1.2 PHP Functions - Adding parameters

To add more functionality to a function, we can add parameters. A parameter is just like a variable.

Parameters are specified after the function name, inside the parentheses.

Example 1.2

The following example will write different first names, but equal last name:

```
<html>
<body>
<?php
function display_name($fname)
{
echo "My name is ". $fname ;
}
display_name (“Zafar”);
?>
</body>
</html>
```

Output:

```
My name is Zafar
```

Example 1.3

The following function has two parameters:

```
<html>
<body>
<?php
function display_name($fname,$lname)
{
echo "My name is ". $fname . ' '. $lname;
}
display_name ("Zafar","Malik");
?>
</body>
</html>
```

Output:

```
My name is Zafar Malik.
```

1.3 PHP Functions - Return values

To let a function return a value, use the return statement.

Example 1.4

```
<html>
<body>
<?php
function add($x,$y)
{
$total=$x+$y;
return $total;
}

echo "1 + 16 = " . add(1,16);
?>
</body>
</html>
```

Output:

```
1 + 16 = 17
```

2. Using Databases

To use a database or query some table in a database, following are the typical steps to do;

1. Create a connection to the database.
2. If connection successful, write a query statement.
3. Parse this query using connection handler and query statement (note that this step is specific to ORACLE only).
4. Execute query.
5. Fetch the result set generated by the query.
6. Loop over the result set and display data using html and PHP.

2.1 Step 1 (Create a Connection to the database)

In PHP, this is done with the [db type]_connect() function.

[db type]_connect(servername, username, password);

Parameter	**Description**
servername	Specifies the server to connect to.
username	Specifies the username to log in with. Default value is the name of the user that owns the database server
password	Specifies the password to log in with.

Example 2.1

Create a web page (blank page) with the name as dbcon.php in your htdocs/training/lecture4/ directory.

Type the following code in the file and run it in the browser.

```
<html>
<body>
<?php
// example 2.1 ..creating a database connection
$db_sid = "(DESCRIPTION = (ADDRESS_LIST = (ADDRESS = (PROTOCOL = TCP)(HOST = zafarlaptop)(PORT = 1521)) )
(CONNECT_DATA = (SID = ORCL) ) )";

$db_user = "scott";
$db_pass = "tiger";

$con = oci_connect($db_user,$db_pass,$db_sid);

if($con)
{
            echo "connection successful.";
}
else
{
             die('Could not connect: ');
}
?>
</body>
</html>
```

In this example we store the connection in a variable ($con) for later use in the script. The "die" part will be executed if the connection fails.

So now if the connection is successful, we'll now proceed to query the EMP table as follows;

2.2 Step 2 (Write a query)

Example 2.2

Write the bold line in your file as follows;

```
<html>
<body>
<?php
// example 2.1 ..creating a database connection
$db_sid = "(DESCRIPTION = (ADDRESS_LIST = (ADDRESS = (PROTOCOL = TCP)(HOST = zafarlaptop)(PORT = 1521)) )
(CONNECT_DATA = (SID = ORCL) ) )";

$db_user = "scott";
$db_pass = "tiger";
```

```
$con = oci_connect($db_user,$db_pass,$db_sid);

if($con)
{
		echo "connection successful.";
}
else
{
		 die('Could not connect: ');
}

$q = "select * from emp";

?>
</body>
</html>
```

2.3 Step 3 (Parse a query)

Prepares SQL (Oracle) statement for execution.

resource **oci_parse** (resource connection, string query);

Parameter	Description
connection	An Oracle connection identifier.
query	The SQL query.

Prepares the `query` using `connection` and returns the statement identifier on success, or **`FALSE`** on error.

Note that This function *does not* validate query. The only way to find out if query is valid SQL statement - is to execute it.

Example 2.3

Write the bold line in your file as follows;

```
<html>
<body>
<?php
// example 2.1 ..creating a database connection
$db_sid = "(DESCRIPTION = (ADDRESS_LIST = (ADDRESS = (PROTOCOL = TCP)(HOST = zafarlaptop)(PORT = 1521)) )
(CONNECT_DATA = (SID = ORCL) ) )";

$db_user = "scott";
$db_pass = "tiger";

$con = oci_connect($db_user,$db_pass,$db_sid);

if($con)
{
		echo "connection successful.";
}
else
{
		 die('Could not connect: ');
```

```
}

$q = "select * from emp";

$query_id = oci_parse($con, $q);

?>
</body>
</html>
```

2.4 Step 4 (Execute a query)

Executes a statement.

bool **oci_execute** (resource statement [, int mode]);

Parameter	Description
Statement	A valid OCI statement identifier.
mode	Allows you to specify the execution mode (default is OCI_COMMIT_ON_SUCCESS). If you don't want statements to be committed automatically, you should specify OCI_DEFAULT as your mode. When using OCI_DEFAULT mode, you're creating a transaction. Transactions are automatically rolled back when you close the connection, or when the script ends, whichever is soonest. You need to explicitly call oci_commit() to commit the transaction, or oci_rollback() to abort it.

Returns **TRUE** on success or **FALSE** on failure.

Example 2.4

Write the bold line in your file as follows;

```
<html>
<body>
<?php
// example 2.1 ..creating a database connection
$db_sid = "(DESCRIPTION = (ADDRESS_LIST = (ADDRESS = (PROTOCOL = TCP)(HOST = zafarlaptop)(PORT = 1521)) )
(CONNECT_DATA = (SID = ORCL) ) )";

$db_user = "scott";
$db_pass = "tiger";

$con = oci_connect($db_user,$db_pass,$db_sid);

if($con)
{
        echo "connection successful.";
}
else
{
         die('Could not connect: ');
}
```

```
$q = "select * from emp";

$query_id = oci_parse($con, $q);

$r = oci_execute($query_id);


?>
</body>
</html>
```

2.5 Step 5 (Fetch the result set)

Returns the next row from the result data as an associative or numeric array, or both.

array **oci_fetch_array** (resource statement [, int mode]);

Parameter	Description
Statement	A valid OCI statement identifier.
mode	An optional second parameter can be any combination of the following constants: OCI_BOTH - return an array with both associative and numeric indices (the same as OCI_ASSOC + OCI_NUM). This is the default behavior. OCI_RETURN_NULLS - create empty elements for the NULL fields.

Returns an array with both associative and numeric indices, or FALSE if there are no more rows in the statement.

Example 2.5

Write the bold line in your file as follows;

```
<html>
<body>
<?php
// example 2.1 ..creating a database connection
$db_sid = "(DESCRIPTION = (ADDRESS_LIST = (ADDRESS = (PROTOCOL = TCP)(HOST = zafarlaptop)(PORT = 1521)) )
(CONNECT_DATA = (SID = ORCL) ) )";

$db_user = "scott";
$db_pass = "tiger";

$con = oci_connect($db_user,$db_pass,$db_sid);

if($con)
{
            echo "connection successful.";
}
else
{
             die('Could not connect: ');
}

$q = "select * from emp";
```

```
$query_id = oci_parse($con, $q);

$r = oci_execute($query_id);

$rs_arr = oci_fetch_array($query_id, OCI_BOTH+OCI_RETURN_NULLS);
?>
</body>
</html>
```

2.6 Step 6 (Loop over the result set)

While loop is best to display result using fetch array method.

Example 2.6

Write the bold line in your file as follows;

```
<html>
<body>
<?php
// example 2.1 ..creating a database connection
$db_sid = "(DESCRIPTION = (ADDRESS_LIST = (ADDRESS = (PROTOCOL = TCP)(HOST = zafarlaptop)(PORT = 1521)) )
(CONNECT_DATA = (SID = ORCL) ) )";

$db_user = "scott";
$db_pass = "tiger";

$con = oci_connect($db_user,$db_pass,$db_sid);

if($con)
{
        echo "connection successful.";
}
else
{
         die('Could not connect: ');
}

$q = "select * from emp";

$query_id = oci_parse($con, $q);

$r = oci_execute($query_id);

$rs_arr = oci_fetch_array($query_id, OCI_BOTH+OCI_RETURN_NULLS);
//print_r($rs_arr);
//echo"<hr>";

while($row = oci_fetch_array($query_id, OCI_BOTH+OCI_RETURN_NULLS))
{
        echo $row['EMPNO'] ."<br>";
}
?>
</body>
</html>
```

The output would be;

```
7521
7566
7654
7698
7782
7788
7839
```

7 Resources

7.1 websites

http://www.w3schools.com/

7.2 Books

Professional PHP Programming by Wrox word press Ltd. **ISBN:** 81-7366-201-0